

VOLODYMYR STETSENKO

Independent Security Researcher

RISC Zero

Ethereum Mainnet Verifiers Security Review

Original assessment: 31 May – 2 June 2026

Report date: 14 October 2026

Independent Security Research Report

Contents

Executive Summary 2

Scope & System Overview 3

Source & State Verification 4

Methodology & Test Results 5

I-01 | Release-Tag Parameter Mismatch 6

I-02 | Proof Bytes Are Not Statement Identity 7

Supporting Technical Observations 8

Conclusions & Limitations 9

References 10

0	0	0	0	2
Critical	High	Medium	Low	Informational

Executive Summary

This independent security review examines selected RISC Zero Ethereum verifier deployments and their proof-validation interfaces. The assessment focuses on receipt-claim binding, version dispatch, deployment parameters, and the boundary between a cryptographic proof and the application that consumes it.

Two Informational observations are documented. The v2.2.0 deployment's control root differs from the constant in the corresponding release tag. Separately, a valid Groth16 proof can be transformed into a byte-distinct proof of the same statement. Neither observation establishes acceptance of a false receipt claim or unauthorized asset movement in the reviewed contracts.

ID	Observation	Severity
I-01	Release-tag control root differs from the deployed v2.2.0 parameter	Informational
I-02	Proof bytes are not a unique identifier for a verified statement	Informational

No Critical, High, Medium, or Low finding is assigned within this focused scope. This is a statement about the evidence obtained, not a certification of the complete proof system.

Validation outcome

On 14 October 2026, thirteen fork tests passed at Ethereum block **26,192,040**, with **zero failures and zero skipped tests**. The original v3.0.0 receipt and its sign-flipped variant both verified, including through the router. Tested changes to the journal digest and image ID were rejected. Malformed and selected invalid proof inputs were also rejected.

Pinned read-only queries confirmed the v2.2.0 and v3.0.0 control-root halves, selectors, wrapper targets, and unpaused wrapper state. Runtime bytes of the router, both Groth16 verifier deployments, and the set verifier matched the original saved bytecode files. This comparison does not establish a complete reproducible source build.

Recommended actions

1. Publish deployment-specific build and parameter records rather than relying on a release label alone.
2. Bind application replay protection to the intended action or statement, including its relevant domain and nonce, rather than to the serialized proof bytes.
3. Preserve regression tests for valid proofs, altered statements, and equivalent proof encodings.

No protocol-team remediation or acknowledgment is claimed.

Scope & System Overview

Deployment scope

Ethereum mainnet, chain ID **1**. Addresses identify the deployments examined; this is not an enumeration of every RISC Zero deployment or every route currently available.

Component	Address
Verifier router	`0x8EaB2D97Dfce405A1692a21b3ff3A172d593D319`
Groth16 verifier v3.0.0	`0x2a098988600d87650Fb061FfAff08B97149Fa84D`
Groth16 verifier v2.2.0	`0xafB31f5b70623CDF4b20Ada3f7230916A5A79df9`
Set verifier v0.9.0	`0x5005aBa3DFf7C940fcc1e48DccCAD611a80eEB85`

Verification flow

The router selects a verifier using the seal's four-byte prefix. The reviewed Groth16 routes point to emergency-stop wrappers, which forward verification to their configured implementations when unpaused. The implementation reconstructs the claim digest, checks the selector, decodes the proof, and invokes the pairing verifier with the deployment's parameters and the claim's digest halves.

The set verifier validates inclusion against an aggregated root. Its `VERIFIER` points to the router, so validation of a supplied root proof crosses the same version-dispatch boundary. An accepted proof establishes a statement; it does not itself authorize a consumer's payment, withdrawal, or other business action.

Source boundary

Original source context: `risc0/risc0-ethereum` at `3aa137844818f0f44e6b2962b6eb958f26d04be7`. Reviewed components include `IRiscZeroVerifier.sol`, `StructHash.sol`, `Util.sol`, `RiscZeroVerifierRouter.sol`, `RiscZeroVerifierEmergencyStop.sol`, `RiscZeroSetVerifier.sol`, and the Groth16 wrapper, generated pairing verifier, and control parameters. Rust aggregation encoding was inspected in `crates/aggregation/src/lib.rs` at the same revision.

Deployment-specific comparisons use the v3.0.0 and v2.2.0 revisions identified in Source & State Verification. The working source context contains a later version label and is not presented as the exact source revision of every deployed contract.

Exclusions and trust assumptions

The zkVM execution circuits, recursive proof-system soundness, trusted-setup integrity, prover implementation as a whole, governance-key compromise, other networks, and downstream consumer contracts are outside this focused review. The assessment assumes Ethereum's specified precompile behavior and the underlying cryptographic assumptions. It does not independently prove them.

Source & State Verification

Pinned state

Block: 26,192,040
Block timestamp: 14 October 2026, 16:48:59 UTC
Block hash: `0xde29c3eb27bf5e74d0e67c920a5488b38d921ea66b44d602a6a23328dcacadb0`

All new fork tests and deployment queries in this report use this block. The original research dates remain 31 May-2 June 2026; a historical fork block was not recorded in the supplied report, so the new results are not described as a replay of an exactly pinned original state.

Deployment	Selector	Observed route
v3.0.0	`0x73c457ba`	Wrapper `0x9F9994Eb4Cb520 0198FEfb470f8b50301662e696`
v2.2.0	`0xbb001d44`	Wrapper `0x1efDd13f831cee Ea14940806705A53D3211CD698`
Set verifier	`0x242f9d5b`	Its `VERIFIER` getter returns the scoped router

Both Groth16 wrappers returned `paused() = false` and the expected implementation address. The set verifier selector and dependency were read directly; its full inclusion-proof path was not rerun in the focused fork suite.

Source revisions

Reference	Full commit
v3.0.0 tag	`32aa0b6f23ddd02dd93fc71717667606e5c7db86`
v2.2.0 tag	`382d76a8040068243a5d13e3de50fcca4224b0d6`
Matching historical v2.2.0 control-root source	`4666fad6c8b6ffe0601703765d7d102f0fe54f23`

The v3.0.0 wrapper's verification logic matches the inspected working-context file; the observed difference is its version string. Source locations for that wrapper in this report refer to the v3.0.0 tag. A matching constant in a historical revision establishes a source match for that constant, not the exact deployment build commit.

Runtime code for the four listed deployments was retrieved and compared with the original saved bytecode. All four comparisons matched. Immutable getter values were checked independently. Constructor transactions, compiler metadata, and complete bytecode reproduction from source were not reconstructed in this revalidation.

Methodology & Test Results

The review combines source inspection, deployment-parameter queries, comparison of saved and pinned runtime code, and assertion-based fork tests. Execution occurred on a local fork using read-only public RPC access; no transaction was broadcast to Ethereum.

Test tooling: Foundry 1.7.1; Solidity 0.8.26; optimizer enabled with 10,000 runs; via-IR; Cancun EVM target. These describe the test harness, not verified production build settings. The harness was changed to fail when target bytecode is absent rather than silently skip.

Test group	Checks	Result
Original receipt and nonzero router route	2	Passed
Selector mutation, zero proof points, altered journal	3	Rejected as expected
Public-signal scalar-field boundary	1	Expected boundary behavior
Selected out-of-range coordinates and truncated seal	3	Rejected as expected
Sign-flipped proof, direct verifier	1	Same statement accepted
Original and sign-flipped proof through router	1	Both accepted
Sign-flipped proof with altered journal; altered image ID	2	Rejected as expected
Total	**13**	**13 passed; 0 failed; 0 skipped**

The known-good receipt is the supplied `TestReceiptV3_0.sol` fixture. The mutation changes the G1 A-point's y coordinate and both G2 B-point y limbs to their additive inverses. It changes the proof bytes without changing the claimed image ID or journal digest.

Interpretation limits

Negative tests sample specific inputs; they do not prove rejection of every invalid claim. The field-edge test compares `r`, `r+1`, and `r-1` using otherwise invalid proof points. Its gas differential is consistent with early rejection, but is not a production gas benchmark or a stand-alone proof of the exact internal failure location. Source inspection provides the complementary evidence.

The `A.y + q` test rejects one specific mutated valid fixture. This result does not establish a universal absence of alternative noncanonical encodings. The genuine sign-flip behavior is assessed separately in I-02.

No formal verification, exhaustive fuzzing campaign, or complete verification of the zkVM and recursion circuits is claimed. No new v2.2.0 valid-receipt test or set-inclusion end-to-end test is included in the thirteen-test result.

I-01 | Release-Tag Parameter Mismatch

Severity: Informational
Category: Deployment documentation and reproducibility
Status: Observed; remediation not assessed
Location: `contracts/src/groth16/ControlID.sol`, line 24, at the v2.2.0 tag and historical revision `4666fad6`; deployed v2.2.0 immutable getters.

Description and evidence

The v2.2.0 deployment exposes two 128-bit control-root halves. Reversing the bytes in each half and concatenating them reconstructs its control root:

Evidence	Value
`CONTROL_ROOT_0`	`0x1b0da5b86dcff31a0242380356bf52ce`
`CONTROL_ROOT_1`	`0x29155a2c00cffdc6221a4fa325c13575`
Reconstructed deployed root	`0xce52bf56033842021af3cf6db8a50d1b7535c1 25a34f1a22c6fdcf002c5a1529`
Root at the v2.2.0 tag	`0x884389273e128b32475b334dec75ee619b77cb 33d41c332021fe7e44c746ee60`

The deployed value matches the constant at historical revision `4666fad6c8b6ffe0601703765d7d102f0fe54f23`. The release tag contains a different value. [R1, R2]

ControlID.sol, v2.2.0 tag, line 24:

```
24      bytes32 public constant CONTROL_ROOT = hex"884389273
         e128b32475b334dec75ee619b77cb33d41c332021fe7e44c746ee60";
```

Impact and rating rationale

A developer using the release label alone can select parameters that do not describe the reviewed deployment, complicating reproduction and integration. No false proof acceptance or deployed verification-key corruption was demonstrated. A source-tag discrepancy is therefore classified as Informational, not as a proof-system soundness vulnerability.

Recommendation

Publish an address-specific manifest containing the exact source commit, compiler settings, constructor parameters, verification-key identity, and runtime hash. Preserve existing tags and document historical differences rather than silently moving a release tag. Validate that consumer configuration is derived from the intended deployment parameters.

I-02 | Proof Bytes Are Not Statement Identity

Severity: Informational
Category: Consumer integration and replay protection
Status: Behavior confirmed; no affected consumer demonstrated
Location: `RiscZeroGroth16Verifier.sol`, lines 155-181, at v3.0.0; pairing-input construction in `Groth16Verifier.sol`.

Description

The verifier accepts a valid Groth16 proof `(A, B, C)` and its sign-flipped representation `(-A, -B, C)` for the same image ID and journal digest. Bilinearity preserves the pairing contribution when both points are negated. This is a proof-representation property, not evidence that an attacker can prove a different execution result.

The wrapper checks the selector and then passes the decoded proof to the pairing verifier. It does not impose proof-byte uniqueness:

RiscZeroGroth16Verifier.sol, v3.0.0, lines 159-169:

```
159         if (SELECTOR != bytes4(seal[:4])) {
160             revert SelectorMismatch({received: bytes4(seal[:4]), expected: SELECTOR});
161         }
162
163         // Run the Groth16 verify procedure.
164         (bytes16 claim0, bytes16 claim1) = splitDigest(claimDigest);
165         Seal memory decodedSeal = abi.decode(seal[4:], (Seal));
166         bool verified = this.verifyProof(
167             decodedSeal.a,
168             decodedSeal.b,
169             decodedSeal.c,
```

Evidence and impact

The fork suite verified that the transformed seal differs from the original bytes and is accepted for the original statement. Both representations also passed through the router. The transformed proof with an altered journal digest was rejected. [R3]

A consumer that uses only `keccak256(seal)` as its spent identifier could treat equivalent proofs as different authorizations. This is conditional impact: no deployed consumer with that design was identified or exploited in this assessment. The verifier's observed behavior alone does not establish a replay vulnerability in a particular application.

Recommendation

Define uniqueness around the authorized business action: bind the statement to the relevant chain, contract, recipient, amount, and nonce or nullifier where required. Persist consumption of that action identifier. Where a statement itself is intended to be single-use, use its authenticated digest rather than the proof encoding. Include equivalent-proof test cases in consumer regression suites. Do not assume that proof canonicalization alone supplies application-level replay protection.

Supporting Technical Observations

Claim and aggregation encoding

The inspected claim-hashing code orders the receipt's input, pre-state, post-state, and output digests, followed by little-endian exit-code words and the down-list length. `SystemState` uses a byte-reversed program counter and the state root; `Output` binds journal and assumptions digests. These observations describe the inspected encoding paths, not an exhaustive proof of equivalence for all Rust and Solidity APIs. [R4]

The aggregation journal uses the image ID followed by the finalized MMR bitmap and root. The Solidity construction is:

RiscZeroSetVerifier.sol, lines 133-136:

```
133     function _encodeJournal(bytes32 root) internal view returns (bytes memory) {
134         // The journal is the encoding of the guest state for a finalized Merkle mountain range.
135         // See GuestState in the risc0-aggregation crate for more information.
136         return abi.encodePacked(IMAGE_ID, uint256(1 << 255), root);
```

The Rust `GuestState::encode` prepends the image ID to `MerkleMountainRange::encode`; the latter uses a big-endian bitmap followed by peak digests. For a finalized single peak, this agrees with the inspected 96-byte layout. [R5]

Leaf hashing includes the eight-byte `LEAF_TAG`, producing a 40-byte leaf preimage. Internal Merkle-node preimages contain two 32-byte values. This distinguishes the direct leaf/internal-node encodings under the hash-security assumption; it is not an unrestricted proof against all Merkle-related attacks.

Pairing and field checks

The generated verifier checks public signals against the BN254 scalar-field modulus. Ethereum's EIP-197 specifies input validity and subgroup requirements for the pairing precompile; the report does not attribute those requirements to a particular execution-client library or later upgrade. [R6]

For the tested proof, adding the base-field modulus to A's y coordinate was rejected. The expression `mod(sub(q, y), q)` performs a 256-bit wrapping subtraction before reduction. It should not be simplistically equated with arithmetic modulo q at every intermediate step. This rejected mutation does not negate the separately confirmed sign-flip property.

Conclusions & Limitations

The evidence supports two Informational observations with different purposes: I-01 concerns deployment reproducibility, while I-02 concerns the contract between a proof verifier and its consumers. The focused test results do not establish a false-claim acceptance path in the reviewed v3.0.0 deployment.

Remediation verification criteria

- For I-01, an address-specific build record should reproduce the observed parameters and identify the associated source and compiler configuration.
- For I-02, an application should reject a second use of the same authorized action even when presented with a byte-distinct equivalent proof, while allowing intentionally distinct actions.
- Any future severity reassessment should identify concrete prerequisites, an affected component, and demonstrated impact. A passing test for an expected cryptographic property is not by itself a security exploit.

Assessment limitations

The October checks revalidate selected observations from the original research. They do not constitute a fresh protocol-wide audit. The review does not establish complete source-to-runtime reproduction, full current route enumeration, production consumer safety, or trusted-setup integrity. Snapshot observations may differ from later chain state.

A finite test suite cannot prove universal soundness. The test harness and raw verification records are retained separately; the report presents the tested conditions and outcomes without embedding the complete harness.

Disclaimer

This report is an independent technical assessment of the identified material. It does not imply a commissioned engagement, certification, or endorsement by RISC Zero. Conclusions apply only to the scope, assumptions, and state described here.

A security review cannot guarantee the absence of vulnerabilities. The report is provided for technical information without warranty and is not legal or investment advice. Source excerpts remain subject to their original notices and licenses. No remediation status is inferred from the absence of a response or from the publication of this document.

References

R1. v2.2.0 release parameters. ControlID.sol at 382d76a.

R2. Historical matching parameter. ControlID.sol at 4666fad6. Matching this constant does not establish the complete deployment source revision.

R3. v3.0.0 verification path. RiscZeroGroth16Verifier.sol and Groth16Verifier.sol.

R4. Receipt hashing. IRiscZeroVerifier.sol and StructHash.sol.

R5. Aggregation encoding. RiscZeroSetVerifier.sol and Rust aggregation library.

R6. Pairing precompile specification. EIP-197.

Deployment references

- Ethereum verifier router
- Groth16 v3.0.0 deployment
- Groth16 v2.2.0 deployment
- Set verifier deployment

All chain observations in the focused revalidation refer to the pinned block in Source & State Verification. Repository references use immutable commits rather than a moving branch.



Volodymyr Stetsenko

Independent Security Researcher

Independent security research focused on smart contracts and decentralized protocols. The work combines manual source analysis, adversarial testing, and deployment verification to produce clear, evidence-based findings and practical recommendations.
This report presents independently conducted research with explicit scope, documented observations, and measured impact assessments.

GitHub / VolodymyrStetsenko

LinkedIn / Volodymyr Stetsenko

X / volodymyr_sec

Independent Security Research Report

© 2026 Volodymyr Stetsenko. All rights reserved.